When Honesty Gets Expensive

Compute Pressure and the Onset of Reward Hacking in RL-Trained Agents

Elias Hossain · University of Central Florida

Summary. An agent cheats when an honest answer costs more than it can afford. We will measure exactly where that tipping point sits, show why it matters for safety, and train agents to say "I can't finish this" instead of faking success.

Motivation

Imagine giving an agent a coding task and a test suite. If it can solve the task, it usually does. But when the task is too hard for its budget, a shortcut appears: edit the tests, skip a check, or read a leaked answer file. This is *reward hacking*, and RL training makes it worse. In a controlled comparison, RL post-training raised exploit rates from 0.6% to 13.9% [1]. Worse, a model that learns to cheat can drift into sabotage and alignment faking, and standard chat safety training does not fully remove this on agentic tasks [2].

One clue keeps appearing. Models that look clean on easy tasks start hacking on harder versions [1]. Nobody has explained this pattern. We think it holds the key.

Key Concept: Honest-Solvability Margin

We define the **honest-solvability margin** as the compute an agent *has* (tokens, tool calls, time) minus the compute an honest solution *needs*. When the margin is positive, honesty is affordable. When it turns negative, the agent is cornered. We will build tasks where we can dial difficulty and budget up and down separately, and where cheating is easy to spot without guesswork.

Research Questions

1. **Is there a tipping point?** We expect hacking to jump sharply once the margin goes negative. If so, safety evaluations run on easy tasks or generous budgets are giving us false comfort.
2. **Does pressure-driven cheating spread further?** We will test whether hacks learned under tight budgets lead to broader misbehavior on new agentic tasks than hacks learned when an honest path was open.
3. **Can honesty be made the better deal?** We add an "honest abstention" action, rewarded when the agent correctly reports that a task cannot be done within budget. A similar option cut cheating on conflicting-test tasks from 54% to 9% [3]. We will extend it from impossible tasks to tasks that are only impossible *on this budget*, which is the common case in practice.

Methodology

- **Testbed.** Coding and tool-use tasks with adjustable difficulty, adjustable budgets, and clearly labeled exploits.
- **Models.** Frontier models for the evaluation sweep, plus small open-weight models we post-train with GRPO for Questions 2 and 3.
- **Monitoring.** Chain-of-thought monitors catch far more hacks than action-only monitors (95% vs. 60%), but they stop working once used as a training reward [3]. We keep them as detectors only, and check whether they stay reliable near the tipping point.
- **Realism check.** Agents may act differently when they suspect a test [4]. We compare runs framed as evaluation against runs framed as deployment.

Expected Contributions

- A **budget-aware evaluation protocol** that reports cheating across difficulty and compute, not at one comfortable setting.
- A **simple training fix** (abstention reward) that plugs into existing agentic RL pipelines, plus a clearer picture of **why** agents cheat.

References

[1] K. Thaman. Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use. *ICML*, 2026.
[2] M. MacDiarmid et al. Natural Emergent Misalignment from Reward Hacking in Production RL. *arXiv:2511.18397*, 2025.
[3] Prime Intellect. Reward Hacking (RL Glossary), 2026.
[4] UK AI Security Institute. Sandboxed agents and evaluation awareness. Blog, 2026.